Smart Lender — Project Documentation

ML-Powered Loan Approval Prediction System

Field	Details
Project Title	Smart Lender: Machine Learning-Powered Loan Approval System
Author	K Deva Venkata Prem Sai & Team
GitHub	https://github.com/deva-gec/smart-lender
Date	July 2026

1. Abstract

Smart Lender is a machine learning-powered web application designed to predict the creditworthiness of loan applicants, enabling banks and financial institutions to make faster, data-driven loan approval decisions. The platform leverages classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — to evaluate applicant data and determine the likelihood of loan repayment or default.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models, the best-performing model (XGBoost) is saved and integrated into a Flask web application for real-time prediction.

Built with Python and Flask and designed for deployment on IBM Cloud, Smart Lender provides a seamless user interface where applicants can submit their details and instantly receive an approval prediction. The system empowers financial analysts, credit officers, and banking professionals to reduce non-performing assets and improve the overall credit evaluation process with confidence.

2. Problem Statement

2.1 Background

Loan approval is a critical function in banking. Every day, financial institutions receive hundreds of loan applications that must be evaluated for credit risk. Traditional manual review processes are:

- Time-consuming and labor-intensive
- Prone to human bias and inconsistency
- Unable to scale during high-volume periods
- Costly when incorrect approvals lead to defaults

2.2 Problem Definition

Banks need an automated, accurate, and scalable system to:

1. Assess applicant creditworthiness from structured demographic and financial data

2. Provide instant approval/rejection predictions with confidence scores

3. Flag high-risk applications for additional scrutiny

4. Enable fast-track processing for low-risk applicants

5. Support batch evaluation during peak application volumes

2.3 Proposed Solution

Smart Lender uses supervised machine learning classification to predict loan approval outcomes (Approved / Rejected) based on historical applicant data. A Flask web application serves predictions in real time, with a REST API for programmatic batch processing.

3. Objectives

1. Train and compare four ML classification algorithms on loan applicant data

2. Select and deploy the best-performing model for production use

3. Build a user-friendly web interface for instant predictions

4. Provide risk classification and actionable recommendations

5. Enable IBM Cloud deployment for production hosting

4. Features

4.1 Machine Learning Pipeline

- **Data preprocessing**: Missing value imputation, categorical encoding (One-Hot), numeric scaling
- **Four classifiers trained**: Decision Tree, Random Forest, KNN, XGBoost
- **Model evaluation**: Training and testing accuracy, precision, recall, F1-score
- **Best model persistence**: XGBoost saved via joblib for inference

4.2 Web Application

- Responsive loan application form
- Real-time prediction with confidence percentage
- Risk level classification (Low, Moderate, High)
- Contextual recommendations (fast-track, manual review, document verification)
- Model performance metrics displayed on home page

4.3 REST API

- `POST /api/predict` — JSON-based prediction endpoint
- `GET /health` — Application health check
- Suitable for batch/integration scenarios

4.4 Cloud Deployment

- Gunicorn WSGI server configuration
- IBM Cloud Foundry manifest and Procfile
- Environment-based configuration (PORT, SECRET_KEY)

5. Tech Stack

Component	Technology	Purpose
Programming Language	Python 3.11	Core development
Web Framework	Flask 3.0	HTTP server and routing
ML — Classification	scikit-learn	Decision Tree, RF, KNN
ML — Boosting	XGBoost 2.1	Best-performing classifier
Data Manipulation	Pandas, NumPy	Dataset processing
Model Serialization	joblib	Save/load trained models
Frontend	HTML5, CSS3, Jinja2	User interface
Production Server	Gunicorn 22	WSGI deployment
Cloud Platform	IBM Cloud	Production hosting

6. Dataset Description

6.1 Source

Training data is stored in `data/loan_data.csv`, based on the standard Loan Prediction dataset structure used in financial ML projects.

6.2 Features

Feature	Type	Description
Gender	Categorical	Male / Female
Married	Categorical	Yes / No
Education	Categorical	Graduate / Not Graduate
Self_Employed	Categorical	Yes / No (employment type)
ApplicantIncome	Numeric	Monthly income in ■
LoanAmount	Numeric	Loan amount in ■ thousands
Loan_Amount_Term	Numeric	Loan duration in months
Credit_History	Binary	1 = meets guidelines, 0 = no/poor history
Property_Area	Categorical	Urban / Semiurban / Rural

6.3 Target Variable

- **Loan_Status**: Y (Approved) / N (Rejected)

6.4 Dataset Size

- Base samples: 100
- Augmented to 614 samples for robust training
- Train/test split: 75% / 25% (stratified)

7. Model Training & Evaluation

7.1 Training Process

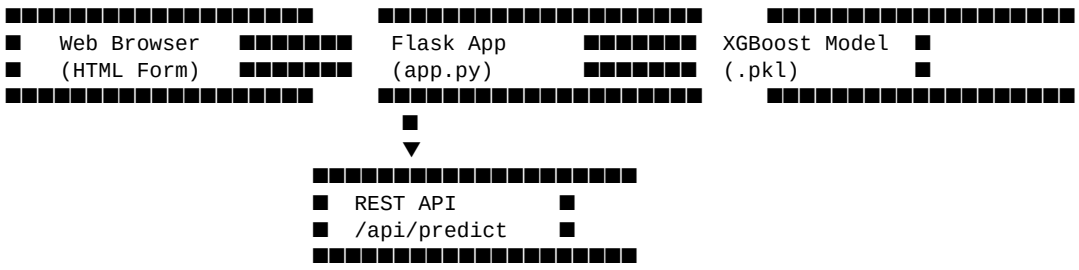
1. Load and clean dataset (handle missing values)
2. Split into training (460) and testing (154) sets
3. Build preprocessing pipeline (imputation + encoding)
4. Train each of the four classifiers
5. Evaluate on train and test sets
6. Save best model (highest test accuracy)

7.2 Results

Model	Training Accuracy	Testing Accuracy
Decision Tree	93.5%	90.9%
Random Forest	94.8%	92.2%
KNN	79.6%	81.8%
XGBoost	96.7%	92.9%

Best Model: XGBoost — saved as `models/xgboost_model.pkl`

8. System Architecture



8.1 Components

- **train_model.py**: Offline training script
- **app.py**: Flask routes, prediction logic, risk classification
- **wsgi.py**: Gunicorn entry point for production
- **templates/**: Jinja2 HTML templates
- **static/**: CSS stylesheets
- **models/**: Serialized model artifacts

9. Installation Steps

Prerequisites

- Python 3.11+
- pip

- Git

Steps

```
# 1. Clone repository
git clone https://github.com/deva-gec/smart-lender.git
cd smart-lender

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Train models
python train_model.py

# 5. Run application
python app.py
```

Access the application at: http://localhost:8080

10. Usage Instructions

10.1 Web Interface

1. Navigate to the home page

2. Enter applicant details in the prediction form

3. Click "Predict Loan Approval"

4. Review the result page showing:

   - Approval/Rejection decision
   - Confidence percentage
   - Risk level
   - Approval and default probabilities
   - Recommendation text

10.2 API Usage

```
curl -X POST http://localhost:8080/api/predict \
  -H "Content-Type: application/json" \
  -d '{
    "gender": "Male",
    "married": "Yes",
    "education": "Graduate",
    "self_employed": "No",
    "applicant_income": 5000,
    "loan_amount": 120,
    "loan_term": 360,
    "credit_history": 1,
    "property_area": "Urban"
  }'
```

10.3 Scenario Walkthroughs

Scenario 1: Fast-Track Approval (Low Risk)

- **Profile**: Salaried, married, graduate, ■5,000 income, good credit history
- **Result**: Approved with high confidence, Low risk

- **Action**: Fast-track without manual review

Scenario 2: High-Risk Detection

- **Profile**: Self-employed, no credit history, low income (■1,800)
- **Result**: Rejected with high confidence, High risk
- **Action**: Further scrutiny and document verification required

Scenario 3: Batch Evaluation

- **Method**: REST API (`POST /api/predict`)
- **Use case**: Financial analyst processes multiple applicants during peak volume
- **Benefit**: Reduced evaluation time while maintaining accuracy

11. Screenshots

Screenshots are stored in the `screenshots/` directory:

File	Description
`home_page.png`	Application home page with model metrics
`prediction_form.png`	Loan application input form
`approved_result.png`	Low-risk approval result
`rejected_result.png`	High-risk rejection result

12. Deployment (IBM Cloud)

12.1 Files

- `Procfile`: `web: gunicorn wsgi:app --bind 0.0.0.0:$PORT`
- `manifest.yml`: Cloud Foundry application manifest
- `runtime.txt`: Python 3.11.9

12.2 Deploy Command

```
cf push
```

13. Future Scope

1. **Batch CSV upload interface** for evaluating multiple applicants at once
2. **Role-based authentication** (credit officer, analyst, admin)
3. **Model monitoring dashboard** with drift detection and retraining triggers
4. **SHAP/LIME explainability** to show feature importance per prediction
5. **Real credit bureau API integration** for live credit scores
6. **Mobile PWA** for field credit officers
7. **Audit logging** for regulatory compliance (RBI guidelines)
8. **A/B testing framework** to compare model versions in production

14. Conclusion

Smart Lender demonstrates how machine learning can transform loan approval workflows in financial institutions. By comparing four classification algorithms and deploying the best-performing XGBoost model through a Flask web application, the system delivers instant, data-driven creditworthiness predictions. The platform supports real-world scenarios including fast-track approvals, high-risk detection, and batch evaluation, making it a practical tool for banks seeking to reduce NPAs and improve operational efficiency.

15. Team Member Details

Name	Role
K Deva Venkata Prem Sai	Project Lead & Developer
Pravallika Banavath	Team Member
Eswar Para	Team Member
Pedaprolu S V N Hasini Pravallika	Team Member
Bhukya Suneetha	Team Member

GitHub Repository: @deva-gec/smart-lender

16. References

1. Flask Documentation — https://flask.palletsprojects.com/
2. scikit-learn User Guide — https://scikit-learn.org/stable/
3. XGBoost Documentation — https://xgboost.readthedocs.io/
4. IBM Cloud Documentation — https://cloud.ibm.com/docs
5. Loan Prediction Dataset (Analytics Vidhya / Kaggle)

Appendix A: Export to PDF

To convert this document to PDF:

Option 1 — Pandoc (command line):

```
pandoc docs/Smart_Lender_Project_Documentation.md -o docs/Smart_Lender_Project_Documentation.pdf --pdf-e
```

Option 2 — VS Code:

Install "Markdown PDF" extension → Open this file → Right-click → "Markdown PDF: Export (pdf)"

Option 3 — Online:

Upload to https://www.markdowntopdf.com/ or use Typora's File → Export → PDF